FlyRank Backend Track — Assignment A9

The Polite Scraper: Ethical Extraction, Schema Validation, & Failure Resilience

Author: Deepak	**Track:** Backend & Data Engineering	**Runtime:** Node.js 20+ / Cheerio / Zod
GitHub: github.com/deepak007679/polite-scraper	**Commits:** 9 Sequential Verified Stages	**Target:** Books to Scrape (Sandbox)

1. Executive Summary & Ethical Principles

Modern AI systems rely on structured, reliable data collection pipelines. A responsible backend scraping pipeline adheres to three foundational tenets: 1. **Check before you collect:** Target classification and robots.txt analysis before writing code. 2. **Be a polite guest:** Identifying User-Agent with repository contact link, request throttling (>=500ms delay), timeout controls, and local caching to minimize server impact. 3. **Trust nothing you scrape:** Web pages are untrusted inputs; every record is normalized and validated against a strict Zod schema before storage.

2. Implementation Stages & Verified Commit History

Stage	Commit Message	Key Deliverables & Actions Taken	Status
Stage 0	Stage 0: classify scraping target	Classified Books to Scrape sandbox; analyzed robots.txt (404); documented ethical terms and 3-page scope.	**PASSED**
Stage 1	Stage 1: fetch and cache HTML	Implemented polite fetcher with honest User-Agent, timeout, and local disk cache in cache/.	**PASSED**
Stage 2	Stage 2: discover three catalogue pages	Crawled 3 catalogue pages dynamically following pagination; extracted 60 unique book URLs via RFC-compliant new URL().	**PASSED**
Stage 3	Stage 3: extract book details	Extracted 8 raw fields per book with Cheerio; enforced provenance receipts (source_page, fetched_at).	**PASSED**
Stage 4	Stage 4: validate normalized records	Normalized price_gbp and ratings; validated against Zod schema; stored 60 idempotent records in output/books.json.	**PASSED**
Stage 5	Stage 5: survive failures, report the run	Isolated page execution; survived deliberate 404 injection without crashing; generated output/run-report.json.	**PASSED**
Stage 6	Stage 6: publish scraper evidence	Comprehensive README with run report evidence, why no browser is needed, limitations, and ethics note.	**PASSED**
Bonus	Bonus: AI vs me	Quarantined AI code in ai-version/; comparative code review on URL resolution, schema rigor, and error reporting.	**PASSED**
Extras	Extras: CSV export, change detection, and unit tests	Added output/books.csv export, SHA-256 change detection across runs, and 5 unit tests in tests/test_parser.js.	**PASSED**

3. Technical Architecture & Engineering Decisions

A. Why No Headless Browser Was Needed

A common pitfall is immediately reaching for Playwright or Puppeteer for all scraping tasks. Because Books to Scrape delivers complete catalogue and product data in static server-rendered HTML, a lightweight HTTP client with Cheerio executes in milliseconds with minimal CPU and memory footprints.

B. Smart Retry Rules & Failure Isolation

Our fetcher handles errors deterministically: • **5xx Server Errors & Timeouts:** Retried once after a 1000ms delay. • **404 Not Found & 403 Forbidden:** Never retried. Requesting a non-existent page or a forbidden resource again is wasteful and impolite. • **Fault Isolation:** Each book is extracted in an isolated try/catch block so a single broken page never terminates the pipeline.

```
// Verified output/run-report.json excerpt
{
"duration_ms": 1284,
"pages_fetched": 0,
"cache_hits": 60,
"valid_records": 60,
"failed_pages": 1,
"failed_urls": [{
"url": ".../deliberately-broken-page-404_9999/index.html",
"reason": "HTTP 404 Not Found - page does not exist"
}]
}
```

C. Bonus Stage Code Review: The AI Rematch

Comparing the hand-built pipeline with the quarantined AI solution in ai-version/ revealed critical trade-offs: 1. **Fragile URL Gluing:** The AI concatenated strings (https://.../${href}) rather than using new URL(). 2. **Dropped Provenance:** The AI stripped source_page and fetched_at receipts. 3. **Missing Telemetry:** The AI skipped writing run-report.json altogether.

4. Submission Artifacts

• **GitHub Repository:** https://github.com/deepak007679/polite-scraper • **Validated JSON Records:** output/books.json (60 unique books) • **CSV Export:** output/books.csv • **Run Telemetry:** output/run-report.json • **Unit Test Suite:** tests/test_parser.js (5 passing tests via npm test)